

MODULE 2 – SELENIUM PYTHON & PYTEST

Assignment Report

Student	Ishani Mandal
Program	B.Tech – Computer Science and Information Technology
Module	Module 2
Technology	Python, Selenium WebDriver and Pytest
Browser	Google Chrome

1. Overview

The automation work done in Module 2 has been covered in this report and it has been done in Python, Selenium WebDriver and Pytest. The assignments are about Page Object Model, reusable page classes, data-driven testing, multiple logins, Assertions, fixtures and HTML test reporting.

The following is an example of an assignment that uses the Page Object Model to automate login.

2. Assignment 7 – Page Object Model Login Automation

Problem: What is the actual problem that you are addressing on your SauceDemo website automation?

Task: Develop an Automation framework for Selenium with Python using Page Classes for Login Page and dashboard page.

Objective: Open SauceDemo and read the login credentials from test data, perform login and verify that the login credentials were entered correctly and the dashboard/inventory page is shown.

Implementation: The test utilizes two classes: LoginPage and DashboardPage. There are individual locators for the name, password and login button. The login() method logs in the user and clicks the login button. The test checks to see if the dashboard is shown, that /inventory.html exists in the current URL and inventory content is available.

Result: Login was successfully automated with Page Object Model.

Key Concept: Page Object Model helps separate page locators and actions from the actual test case, making the automation structure more easily maintained.

3. Assignment 8 – Data-Driven Login Testing

Objective: Test the SauceDemo login functionality with multiple sets of usernames, passwords and expected results.

Implementation: A test-data list was made for different logins such as standard_user, locked_out_user, problem_user, performance_glitch_user and an invalid user. Each test data entry is comprised of user's name, password and expected login result.

A reusable login() function opens SauceDemo, waits for the username and password fields, enters the credentials and clicks the Login button. If the script finds a login success message in the inventory.html URL, it proceeds to the next step; for an unsuccessful login, it checks the corresponding result.

Verification: Each set of login data is tested to see if it passes the test; the result is printed as Test Passed or Test Failed.

End Result: Multiple login scenarios were automated with one reusable test flow.

Key Concept: Data Driven Testing enables the same automation logic to be run against various test data and expected results.

4. Assignment 9 – Pytest Login Testing

Objective: Develop and implement individual Pytest test cases for valid, locked out and invalid logins.

Implementation: A fixture in Pytest is used to create a Chrome WebDriver, maximize the browser and close the browser after the test.

Three test cases were created: Valid Login using `standard_user` and `secret_sauce`; Locked User using `locked_out_user` and `secret_sauce`; and Invalid Login using an invalid username and password.

Verification: Assertions were used to ensure the expected result for each of the login scenarios.

Result: HTML Report showed 3 passed, 0 failed and 0 skipped tests. The three tests were valid login, locked user and invalid login.

Key Concept: Pytest offers fixtures, assertions and structured test execution, and the HTML plugin offers a readable report of the test execution.

5. Page Object Model Structure

The `BasePage` class is a reusable class for the framework of Module 2. It offers functions that are similar to clicking on an element, entering text, retrieving text and getting the current URL. It also includes `WebDriverWait` with waits for 10 seconds.

The `LoginPage` class extends the `BasePage` and contains the locators of the Login Page elements: the username, the password and the Login button.

The `DashboardPage` class contains the inventory container as well as methods to check if the inventory page is shown and to pick up its text.

This is the structure used to keep common Selenium functionality in the base class and page-specific functionality inside their respective page classes.

6. Test Reporting

`pytest-html` was used to create an HTML test report. The report is generated on 24-Sep-2026 and it displays that there were three tests in an execution. All three tests passed with no failures, skips or errors.

The report was for the test execution as follows:

- `test_valid_login` – Passed
- `test_locked_user` – Passed
- `test_invalid_login` – Passed

The environment information listed in the report includes Python 3.14.7, Windows 11 and pytest 9.1.1.

7. Environment and Dependencies

- Python
- Selenium WebDriver
- Pytest
- `pytest-html`
- Google Chrome
- `WebDriverWait`
- Expected Conditions

8. Learning Outcomes

- Created Selenium automation in Python and Pytest.
- Implemented the Page Object Model.
- Wrote reusable Selenium methods and Page classes.
- Used explicit waits and expected conditions.
- Completed login testing based on data with various user credentials.
- Developed individual Pytest test scenarios for various login scenarios.
- Utilised fixtures to set up and tear down the browser.
- Made assertions to check expected application behaviour.

- Created a HTML report of the test results using pytest-html.
- Found out how to structure Selenium automation into reusable components.

9. Summary

During Module 2 we had hands-on experience creating a more structured Selenium automation framework via Python, Selenium and Pytest. Tasks were about implementing Page Object Model, re-usable methods, data-driven testing and different login scenarios.

The module also provided experience with Pytest fixtures, assertions and HTML reporting. The test report that was created successfully marked all three of the Assignment 9 test cases.